



学海拾贝 (/t/notes)

计算机系统原理 - 考前冲刺核心知识点



Chaos (/u/1) 2025年10月25日 已编辑



计算机系统原理 - 考前冲刺核心知识点

第一章：计算机系统概述

1. 冯·诺依曼结构 (Von Neumann Architecture)

核心思想

- 存储程序 (Stored Program)**: 把程序（指令序列）和数据都存放在同一个存储器（内存）里，计算机能自动按顺序执行指令。这是现代计算机的基础。
- 通俗理解**: 就像菜谱（程序）和食材（数据）都放在冰箱（内存）里，厨师（CPU）按菜谱步骤自动取食材做菜。

五大部件

部件	功能描述	通俗类比
运算器 (ALU)	负责算术（加减乘除）和逻辑（与或非）运算	大脑的计算部分
控制器 (CU)	指挥计算机各部件协调工作，负责取指令、分析指令、发出控制信号	大脑的总指挥
存储器	存放程序和数据的地方，主要指内存（主存）	大脑的短期记忆
输入设备	把外界信息（如键盘敲击、鼠标点击）转换成计算机能识别的二进制信息	计算机的“耳朵”“眼睛”
输出设备	把计算机处理的结果（二进制信息）转换成人类能看懂的形式（如屏幕显示、打印）	计算机的“嘴巴”“双手”

关键补充

- 二进制表示**: 计算机内部所有信息（指令和数据）都用二进制（0 和 1）表示。
- 指令结构**: 指令通常由 **操作码 (Opcode)** 和 **地址码 (Address)** 组成:
 - 操作码: 指示计算机执行什么操作（如加法、读取）。
 - 地址码: 指示操作数（数据）在哪里（如在哪个内存地址或寄存器）。

2. 关键部件详解（重点）

(/)

- **CPU (中央处理器)**: 由运算器 (ALU)、控制器 (CU) 及部分寄存器 (Registers) 组成, 是计算机的核心, 负责执行指令。
- **主存 (Main Memory)**: 存放正在运行的程序和数据, CPU 可直接访问; 特点是速度较快但断电信息丢失 (易失性)。
- **寄存器**: CPU 内部的高速存储单元, 用于临时存放指令、数据、地址等, 比内存快得多, 核心类型如下:
 - **PC (程序计数器)**: 存放下一条要执行指令的地址, CPU 靠它自动顺序执行。
 - **IR (指令寄存器)**: 存放当前正在执行的指令。
 - **MAR (存储器地址寄存器)**: 存放将要访问的主存单元的地址。
 - **MDR (存储器数据寄存器)**: 临时存放从主存读取或要写入主存的数据。
 - **通用寄存器 (GPRs)**: 存放操作数或运算结果。
- **总线 (Bus)**: 连接计算机各部件的信息传输线, 包括地址总线、数据总线、控制总线。

3. 程序和指令的执行过程

- **指令 (Instruction)**: 指示 CPU 执行一个基本操作的二进制命令。
- **程序 (Program)**: 一系列指令的集合, 用于完成特定任务。
- **执行过程 (指令周期)**: CPU 执行一条指令通常分为以下步骤:
 1. **取指令 (Fetch)**: CPU 根据 PC 中的地址从主存读取指令到 IR。
 2. **译码 (Decode)**: 控制器分析 IR 中的指令, 确定操作类型和操作数地址。
 3. **执行 (Execute)**: 运算器根据指令要求进行运算, 或访问主存读取 / 写入数据。
 4. (可能) **写回 (Write Back)**: 将执行结果写回寄存器或主存。
 5. **PC 更新**: 取指令后, PC 自动指向下一条指令地址 (通常是当前地址 + 指令长度); 若为跳转指令, PC 更新为跳转目标地址。

4. 程序的开发与运行

语言分类

语言类型	特点	示例
机器语言	CPU 能直接识别和执行的二进制代码 (0 和 1), 对人极其不友好	01010101
汇编语言	用助记符 (如 MOV, ADD) 代替二进制操作码, 仍依赖具体机器	MOV %eax, %ebx
高级语言	接近自然语言, 不依赖具体机器, 易于编写和维护	C, Java, Python

翻译程序

- **汇编程序 (Assembler)**: 将汇编语言翻译成机器语言。
- **编译器 (Compiler)**: 将高级语言一次性翻译成汇编语言或机器语言目标程序。
- **解释程序 (Interpreter)**: 将高级语言逐条翻译成机器指令并立即执行。

C 语言从源程序到可执行文件的流程

1. **预处理 (Preprocessing)**: 处理 #include、#define 等预处理指令, 生成 .i 文件。

2. 编译 (Compilation): 将预处理后的代码翻译成汇编代码, 生成 .s 文件。
3. 汇编 (Assembly): 将汇编代码翻译成机器语言 (可重定位目标文件), 生成 .o 文件。
4. 链接 (Linking): 将多个 .o 文件和库文件合并成一个可执行文件。

5. 计算机系统层次结构

- 层次划分 (从上层到下层): 应用层 → 高级语言层 → 汇编语言层 → 操作系统层 → 指令集体系结构 (ISA) 层 → 微体系结构层 → 硬件逻辑层 → 物理器件层。
- 透明性: 在某个较高层次上看不到或不需要关心较低层次的实现细节。
- 关键接口:
 - 指令集体系结构 (ISA): 软硬件之间的接口规范, 定义了指令格式、操作等, 是程序员看到的概念机器。
 - 微体系结构: ISA 的具体实现, 涉及 CPU 内部设计, 对程序员通常透明。

6. 计算机性能评价

核心指标

- 响应时间 (Response Time): 执行一个任务所需的总时间。
- 吞吐率 (Throughput): 单位时间内完成的任务数量。
- CPU 时间: CPU 执行某个程序所花费的时间, 分为用户 CPU 时间 (执行用户代码的时间) 和系统 CPU 时间 (运行操作系统代码的时间), 通常关注用户 CPU 时间。

关键概念与公式

- 时钟周期: CPU 执行最基本操作的时间单位。
- 时钟频率: 时钟周期的倒数, 单位 Hz (如 GHz), 频率越高, 通常速度越快。
- CPI (Cycles Per Instruction): 执行一条指令平均需要的时钟周期数, CPI 越小, 性能越好。
- 核心公式:
 1. CPU 时间 = 指令条数 × CPI × 时钟周期
 2. CPU 时间 = 指令条数 × CPI / 时钟频率
 3. MIPS (每秒百万条指令) = 指令条数 / (执行时间 × 10⁶) = 时钟频率 / (CPI × 10⁶)

第二章：数据的表示和运算

1. 数制与编码

进位计数制基础

- 基数 (Radix): 每个数位上可能使用的符号个数 (如二进制基数是 2, 十进制是 10)。
- 权 (Weight): 每个数位对应的数值大小 (基数的幂)。

常用数制

数制	基数	符号范围	特点
二进制	2	0, 1	计算机内部使用

数制	基数	符号范围	特点
八进制	8	0-7	简化二进制表示
十进制	10	0-9	人类常用
十六进制	16	0-9, A-F	常用于简化二进制表示

数制转换（重要）

- **R 进制转十进制**：按权展开求和。
示例： $(101.1)_2 = 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 + 1 \times 2^{-1} = 4 + 0 + 1 + 0.5 = (5.5)_{10}$
- **十进制转 R 进制**：
 - 整数部分：除基取余，逆序排列。
 - 小数部分：乘基取整，顺序排列。
- **二进制与八 / 十六进制转换**：
 - 二进制转八进制：小数点开始，左右每 3 位一组，不足补 0，按组转换。
 - 八进制转二进制：每 1 位八进制数转换为 3 位二进制数。
 - 二进制转十六进制：小数点开始，左右每 4 位一组，不足补 0，按组转换。
 - 十六进制转二进制：每 1 位十六进制数转换为 4 位二进制数。

2. 定点数表示

- **定点数**：小数点位置固定不变的数（通常约定在最前或最后），分为定点整数和定点小数。
- **机器数**：数值在计算机中的二进制表示形式（带符号位）。
- **真值**：带正负号的实际数值。

编码方式（重要）

编码类型	规则	优缺点
原码	最高位是符号位（0 正 1 负），其余位是数值的绝对值	简单直观；但加减运算复杂，+0 和 -0 表示不唯一（[+0] 原 = 00...0, [-0] 原 = 10...0）
反码	正数反码同原码；负数反码是原码除符号位外各位取反	加减运算比原码简单；但 +0 和 -0 表示仍不唯一（[+0] 反 = 00...0, [-0] 反 = 11...1），较少使用
补码 (核心)	正数补码同原码；负数补码 = 反码 + 1（或原码除符号位外取反 + 1）	+0 和 -0 表示唯一；加减运算统一用加法实现；n 位补码比原码 / 反码多表示一个最小负数，现代计算机整数普遍使用
移码	移码 = 真值 + 偏置值（Bias），偏置值通常取 2^{n-1} 或 $2^{n-1}-1$	便于比较大小（可直接按无符号数比较）；0 的表示唯一，常用于浮点数阶码

补码关键规则

- **n 位补码表示范围**： -2^{n-1} 到 $+2^{n-1} - 1$ （如 8 位补码范围是 -128 到 +127）。
- **求相反数的补码**：对补码连同符号位各位取反 + 1（注意：最小负数没有对应的正数，取反加一后还是自身并溢出）。
- **真值计算**：符号位为 0 则数值位即真值；符号位为 1 则数值位取反加 1，符号为负。

3. 整数表示

(/)

- **无符号整数**：所有位都用于表示数值，没有符号位，仅表示非负数；n 位无符号整数范围：0 到 $2^n - 1$ ，常用于地址运算。
- **带符号整数**：通常用补码表示；n 位带符号整数（补码）范围： -2^{n-1} 到 $+2^{n-1} - 1$ 。
- **C 语言中的整数**：
 - int, short, long 等是带符号整数（默认用补码）。
 - unsigned int, unsigned short 等是无符号整数。
 - 注意：带符号数和无符号数之间的隐式转换可能改变数值含义（二进制位模式不变，解释方式改变）。

4. 浮点数表示 (IEEE 754 标准 - 核心重点)

目的与格式

- **目的**：表示带有小数的实数，且表示范围比定点数大得多。
- **基本格式**： $X = (-1)^s \times M \times R^E$
其中：S（符号位，0 正 1 负）、M（尾数，定点小数）、E（阶码，定点整数）、R（基数，通常是 2）。

IEEE 754 标准规范

精度类型 总位数 符号位 (S) 阶码 (E) 尾数 (M) 偏置值 (Bias) 实际指数范围

单精度	32	1	8	23	127	-126 到 +127
双精度	64	1	11	52	1023	-1022 到 +1023

关键规则

- **规格化**：尾数通常规格化为 1.f（f 是小数部分），整数部分的 1 是“隐藏位”，不存储，提高一位精度。
- **阶码表示**：使用移码，但偏置值特殊（单精度 $127=2^8-1$ ，双精度 $1023=2^{11}-1$ ），实际指数 = 阶码值 - 偏置值。
- **特殊值（重要）**：
 - ± 0 ：阶码全 0，尾数全 0。
 - $\pm \infty$ （无穷大）：阶码全 1，尾数全 0。
 - NaN (Not a Number)：阶码全 1，尾数非 0，表示无效运算结果（如 0/0）。
 - 非规格化数：阶码全 0，尾数非 0，用于表示绝对值非常小的数（接近 0），此时尾数形式为 0.f，无隐藏位，指数固定为最小指数（如单精度 - 126）。

5. 非数值数据编码

- **逻辑值**：用二进制位表示真 (1) 或假 (0)，可按位进行逻辑运算（与、或、非、异或）。
- **西文字符 (ASCII 码)**：
 - 7 位二进制编码，可表示 128 个字符（字母、数字、标点、控制符）。
 - 通常用 1 字节 (8 位) 存储，最高位为 0。
- **汉字字符 (GB2312)**：



- 区位码：每个汉字在 GB2312 表中的行号（区号）和列号（位号）。
- 国标码（交换码）：区位码的区号和位号分别加 20H（十进制 32）。
- 机内码（内码）：国标码两个字节的最高位都置为 1，用于与 ASCII 码区分，是汉字在计算机内部的存储代码。

6. 数据宽度与排列

- 位 (bit, b)：最小信息单位。
- 字节 (Byte, B)：常用基本单位，1 Byte = 8 bits。
- 字 (Word)：CPU 一次处理的数据位数，与机器字长有关（如 16 位、32 位、64 位）。
- 字节序 (Endianness, 重要)：
 - 大端模式 (Big-Endian)：高位字节存放在低地址，低位字节存放在高地址（符合人类阅读习惯）。
 - 小端模式 (Little-Endian)：低位字节存放在低地址，高位字节存放在高地址（Intel x86 架构使用）。
 - 记忆技巧：小端 → 低对低（低位字节对应低地址）。

7. 定点数运算

加法器基础

- 半加器：两个 1 位二进制数相加，不考虑低位进位。
- 全加器 (Full Adder)：两个 1 位二进制数和来自低位的进位信号相加，是构成多位加法器的基本单元。
- 行波进位加法器：n 个全加器串联，低位进位输出连接到高位进位输入，结构简单但速度慢（进位需逐位传递）。
- ALU (算术逻辑单元)：CPU 中执行算术和逻辑运算的部件，核心是加法器。

补码加减法（重要）

- 加法规则： $[X+Y]_{补} = [X]_{补} + [Y]_{补} \pmod{2^n}$ ，符号位参与运算，按二进制加法规则计算。
- 减法规则： $[X-Y]_{补} = [X]_{补} + [-Y]_{补} \pmod{2^n}$ ，通过求 $[-Y]_{补}$ （对 $[Y]_{补}$ 连同符号位取反 + 1）将减法转为加法。

溢出判断（三种方法）

1. 双符号位 / 变形补码：用两位表示符号位，若结果两位符号位不同（01 或 10），则溢出；01 为上溢（正溢），10 为下溢（负溢）。
2. 单符号位：仅当两个同符号数相加（或异符号数相减）时可能溢出；若结果符号位与操作数符号位相反，则溢出。
3. 进位判断：最高数值位的进位 (C_n) 和符号位的进位 (C_s) 不同时（异或为 1），则溢出，公式： $OF = C_s \oplus C_n$ 。

乘除法步骤

1. 阶码运算：乘法→阶码相加，除法→阶码相减（注意减去偏置值的影响，如单精度需先减 127 再运算，最后加 127）。
2. 尾数运算：乘法→尾数相乘，除法→尾数相除（按定点数乘除法规则，符号位单独异或）。

3. 规格化、舍入、判溢出：与加减法步骤 3-5 一致。

第三章：程序的转换及机器级表示

1. 指令集体系结构 (ISA)

- 定义：软件和硬件之间的接口规范，定义程序员可见的机器状态（寄存器、内存）、指令集（指令格式、操作、寻址方式）等。
- 分类对比：

类型	特点	示例
CISC（复杂）	指令数量多、功能复杂、长度可变、寻址方式多、接近高级语言	Intel x86
RISC（精简）	指令数量少、功能简单、长度固定、寻址方式少、仅 Load/Store 指令访存、多通用寄存器	ARM, MIPS, RISC-V

2. IA-32 (x86) 架构基础

数据类型

- 字节 (8 位)、字 (16 位)、双字 (32 位)，对应指令后缀分别为 b、w、l（如 movb、movw、movl）。

核心寄存器

- 通用寄存器（8 个 32 位）：EAX（累加器）、EBX（基址寄存器）、ECX（计数寄存器）、EDX（数据寄存器）、ESI（源变址）、EDI（目的变址）、ESP（栈指针）、EBP（帧指针），可部分访问（如 EAX 的低 16 位为 AX，AX 的低 8 位为 AL，高 8 位为 AH）。
- 指令指针 (EIP)：存放下一条要执行指令的地址，相当于 PC。
- 标志寄存器 (EFLAGS)：存放条件码（CF 进位、ZF 零、SF 符号、OF 溢出等）和控制位，用于条件判断和中断控制。

寻址方式（重要）

寻址方式	格式示例	说明
立即数寻址	movl \$10, %eax	操作数直接在指令中（\$ 后为立即数）
寄存器寻址	movl %ebx, %eax	操作数在寄存器中（% 后为寄存器名）
直接寻址	movl my_var, %eax	指令中包含操作数的内存地址（my_var 为符号地址）
间接寻址	movl (%ebx), %eax	寄存器内容为操作数的内存地址（括号表示间接访问）
基址 + 偏移量	movl 8(%ebp), %eax	基址寄存器（如 EBP）内容 + 偏移量 = 内存地址，常用于访问栈帧参数 / 局部变量
基址 + 变址 + 比例因子	movl 4(%ebx, %esi, 2)	地址 = 基址 (% ebx)+ 变址 (% esi)× 比例因子 (1/2/4/8)+ 偏移量，常用于访问数组

3. 常用指令类型 (AT&T 语法: 源在前, 目的在后)

(/) 数据传送指令

- `movl Src, Dst` : 传送双字数据 (如 `movl %eax, %ebx`) , 注意不能直接从内存到内存。
- `pushl Src` : 压栈, ESP 先减 4, 再将 Src 写入 ESP 指向的栈单元 (如 `pushl %eax`) 。
- `popl Dst` : 出栈, 从 ESP 指向的栈单元读取数据到 Dst, ESP 再加 4 (如 `popl %ebx`) 。
- `leal Src, Dst` : 加载有效地址, 将 Src 的地址计算结果存入 Dst (非加载内存内容) , 常用于计算地址或简单算术 (如 `leal 2 (%eax), %ebx` $\rightarrow$ `%ebx = %eax + 2`) 。

算术运算指令

- `addl Src, Dst` : $Dst = Dst + Src$ (如 `addl %eax, %ebx`) 。
- `subl Src, Dst` : $Dst = Dst - Src$ (如 `subl %eax, %ebx`) 。
- `incl Dst` : $Dst = Dst + 1$ (如 `incl %eax`) ; `decl Dst` : $Dst = Dst - 1$ (如 `decl %ebx`) 。
- `imull Src, Dst` : 带符号乘法, $Dst = Dst \times Src$ (如 `imull %ebx, %eax`) ; `mul` 为无符号乘法。
- `idivl Divisor` : 带符号除法, 被除数在 EDX:EAX (EDX 高位, EAX 低位) , 商存入 EAX, 余数存入 EDX (如 `idivl %ebx`) ; `div` 为无符号除法。
- `cmpl Src1, Src2` : 计算 $Src2 - Src1$, 仅更新 EFLAGS 标志位, 不保存结果 (用于后续条件判断, 如 `cmpl %eax, %ebx`) 。

逻辑运算与移位指令

- `andl Src, Dst` : 按位与 ($Dst = Dst \& Src$) ; `orl` : 按位或; `xorl` : 按位异或; `notl Dst` : 按位取反。
- `shll $k, Dst` : 逻辑左移 k 位 (高位弃, 低位补 0) ; `sarl $k, Dst` : 算术右移 k 位 (高位补符号位, 低位弃) ; `shr $k, Dst` : 逻辑右移 k 位 (高位补 0, 低位弃) 。

控制流指令

- `jmp Label` : 无条件跳转 (如 `jmp loop_start`) ; `jmp *Operand` : 间接跳转 (目标地址在寄存器 / 内存, 如 `jmp %eax`) 。
- `jcc Label` : 条件跳转, 根据 EFLAGS 标志位决定是否跳转 (如 `je` 相等、`jne` 不相等、`jg` 大于、`jl` 小于等, 如 `je exit`) 。
- `call Label` : 函数调用, 将返回地址 (下一条指令地址) 压栈, 跳转到 Label (如 `call func`) 。
- `ret` : 函数返回, 从栈顶弹出返回地址到 EIP (如 `ret`) 。
- `int $0x80` : 软件中断 / 陷阱, 用于系统调用 (如 `int $0x80` 触发 Linux 系统调用) 。

4. C 语言程序的机器级表示 (核心重点)

过程调用 (函数调用)

栈帧结构 (Stack Frame)

- 每个函数调用在栈上分配的区域, 用于存储: 返回地址、旧 EBP (调用者帧指针)、保存的寄存器、局部变量、函数参数 (IA-32 中参数通过栈传递) 。
- 关键寄存器:

<
(/)

- ESP（栈指针）：指向栈顶（低地址方向增长）。
- EBP（帧指针）：指向当前栈帧底部（高地址），用于稳定访问参数和局部变量。

调用过程（步骤）

1. 调用者操作：

- 将函数参数按**从右到左**的顺序压栈（如调用 `func(a,b) → pushl b → pushl a`）。
- 执行 `call func`：将返回地址（下一条指令地址）压栈，跳转到 `func` 入口。

2. 被调用者操作（函数序言）：

- `pushl %ebp`：保存调用者的旧 EBP 到栈。
- `movl %esp, %ebp`：设置当前栈帧的 EBP（新帧指针）。
- `subl $N, %esp`：为局部变量分配 N 字节栈空间（N 为局部变量总大小）。
- 保存被调用者需保留的寄存器（%ebx、%esi、%edi，若使用需压栈）。

3. 执行函数体：运算逻辑，将返回值存入 %eax（IA-32 约定）。

4. 被调用者操作（函数尾声）：

- 恢复被保存的寄存器（如 `popl %edi`、`popl %esi`、`popl %ebx`）。
- `movl %ebp, %esp`（或 `leave` 指令）：恢复 ESP 到 EBP（释放局部变量空间）。
- `popl %ebp`：恢复调用者的旧 EBP。
- `ret`：弹出返回地址到 EIP，跳回调用者。

5. 调用者操作：`addl $M, %esp`：清理栈上的参数（M 为参数总字节数，如 2 个 int 参数则 M=8）。

寄存器使用约定

- **调用者保存寄存器**：%eax、%ecx、%edx，被调用者可随意修改；调用者若需保留这些寄存器的值，需在调用前自行压栈保存。
- **被调用者保存寄存器**：%ebx、%esi、%edi，被调用者若使用这些寄存器，必须在函数序言压栈保存，在函数尾声恢复。

控制结构的机器级实现

- **if-else**：用 `cmpl` 比较 + `jcc` 条件跳转实现，不满足条件则跳转到 `else` 分支，满足则执行 `if` 分支，分支结束后用 `jmp` 跳转到整体结束处。
- **switch**：若 `case` 值连续，常用**跳转表（Jump Table）**实现：跳转表是存储 `case` 代码块地址的数组，通过 `switch` 变量计算索引，查表得到目标地址，用 `jmp *Operand` 间接跳转，效率高于 `if-else` 链。
- **循环（while/for/do-while）**：
 - `while/for`：先通过 `cmpl + jcc` 判断循环条件，不满足则跳出，满足则执行循环体，最后 `jmp` 回条件判断处。
 - `do-while`：先执行循环体，再判断条件，不满足则跳出，满足则 `jmp` 回循环体开头（至少执行一次）。

复杂数据类型

- **数组**：内存中连续存储相同类型元素，地址计算公式： $\&A = \text{数组基地址} + i \times \text{元素大小}$ （如 `int` 数组元素大小 4 字节）；汇编中常用“**基址 + 变址 + 比例因子**”寻址（如 `movl (%ebx, %esi, 4), %eax`，%ebx 为基地址，%esi 为 `i`，4 为 `int` 大小）。

- < (/)
- **结构体 (struct)**：成员按定义顺序连续存储，可能存在**内存对齐 (Padding)**；成员访问通过“结构体基地址 + 成员偏移量”实现（偏移量在编译时确定，如 `movl 8 (%eax), %ebx, %eax` 为结构体指针，8 为成员偏移量）。
 - **联合体 (union)**：所有成员共享同一块内存空间，大小等于最大成员的大小；同一时间仅能存储一个成员的值，访问不同成员会覆盖原有数据。

数据对齐（重要）

- **目的**：提高内存访问效率，CPU 通常要求特定类型数据的地址是其大小的倍数（如 `int`（4 字节）地址需是 4 的倍数）。
- **IA-32 对齐规则**：
 1. `char`（1 字节）：无对齐要求。
 2. `short`（2 字节）：地址是 2 的倍数。
 3. `int`、`float`、指针（4 字节）：地址是 4 的倍数。
 4. `double`（8 字节）：地址是 8 的倍数（特殊，非 4 的倍数）。
- **结构体对齐规则**：
 1. 第一个成员偏移量为 0。
 2. 后续成员偏移量需是自身对齐要求的整数倍（不足则插入填充字节）。
 3. 结构体总大小需是所有成员中“最大对齐要求”的整数倍（不足则在末尾插入填充字节）。

5. x86-64 架构（补充）

- **通用寄存器**：增至 16 个（`RAX`、`RBX`、`RCX`、`RDY`、`RSI`、`RDI`、`RBP`、`RSP`、`R8R15`），均为 64 位宽，低 32 位可单独访问（如 `EAX` 是 `RAX` 的低 32 位）。
- **地址空间**：64 位虚拟地址（实际有效地址通常为 48 位），支持更大内存。
- **参数传递**：前 6 个整型 / 指针参数通过寄存器传递（`%rdi`、`%rsi`、`%rdx`、`%rcx`、`%r8`、`%r9`），第 7 个及以后参数通过栈传递；浮点参数通过 `XMM` 寄存器传递，减少栈操作，提升调用效率。
- **浮点运算**：主要使用 `SSE` 指令集和 `XMM` 寄存器（128 位），替代传统 `x87` 浮点栈。

第四章：可执行文件的生成与加载执行

1. 目标文件格式（ELF - Executable and Linkable Format）

- **Linux/Unix 系统通用格式**，支持可重定位、可执行、共享三种目标文件类型：

文件类型	后缀	生成工具	特点
可重定位目标文件	.o	汇编器 (as)	包含代码和数据，地址从 0 开始，需与其他.o 文件链接后才能执行
可执行目标文件	无（如 a.out）	链接器 (ld)	包含可直接加载的代码和数据，地址为虚拟地址，可独立执行
共享目标文件	.so	编译器 + 链接器	特殊可重定位文件，加载 / 运行时动态链接，多进程可共享

ELF 文件核心结构

- <

(/)
- **ELF 头 (ELF Header)**：文件开头，描述文件类型（可重定位 / 可执行）、机器架构（x86/ARM）、入口点地址、节头表 / 程序头表的偏移量和大小等全局信息。
 - **节 (Sections)**：可重定位文件的基本组成单位，存储特定类型数据：
 - `.text`：机器代码（指令），只读。
 - `.rodata`：只读数据（如常量字符串、`const` 变量）。
 - `.data`：已初始化的全局 / 静态变量（非 `const`）。
 - `.bss`：未初始化或初始化为 0 的全局 / 静态变量，不占文件空间，仅记录大小，加载时分配内存并清零。
 - `.symtab`：符号表，记录模块中定义和引用的符号（变量名、函数名）信息。
 - `.rel.text/.rel.data`：重定位信息，标记代码 / 数据中需要修改的地址（链接时替换为最终地址）。
 - `.strtab`：字符串表，存储符号名、节名等字符串（`.symtab` 通过索引引用此处内容）。
 - **节头表 (Section Header Table)**：描述每个节的名称、类型、大小、在文件中的偏移量、权限等，供链接器使用。
 - **程序头表 (Program Header Table)**：仅存在于可执行文件和共享文件中，描述如何将文件加载到内存（如代码段、数据段的虚拟地址、大小、权限），供加载器使用；一个“段 (Segment)”通常由多个功能相关的“节 (Section)”组成（如 `.text` 和 `.rodata` 合并为代码段）。

通常由多个功能相关的“节 (Section)”组成（如 `.text` 和 `.rodata` 合并为代码段）。

2. 符号和符号解析

- **符号**：程序中的变量名、函数名等标识，是模块间链接的“桥梁”。
- **符号表 (.symtab)**：记录每个符号的属性，包括符号名（索引指向 `.strtab`）、符号类型（全局 / 外部 / 本地）、符号所在节（如 `.text/.data`）、符号地址 / 偏移量、符号大小。

符号分类

符号类型	定义位置	可见范围	示例
全局符号	本模块	本模块及其他模块	非 <code>static</code> 的全局变量、函数
外部符号	其他模块	仅本模块引用	<code>extern</code> 声明的变量 / 函数
本地符号	本模块	仅本模块内部	<code>static</code> 的变量 / 函数

符号解析规则 (Linux 环境)

- **强符号与弱符号**：
 - 强符号：已初始化的全局变量、函数名（不可重复定义）。
 - 弱符号：未初始化的全局变量（可重复定义）。
- **多重定义处理**：
 1. 不允许多个同名强符号（链接错误）。
 2. 一个强符号 + 多个弱符号：以强符号为准。
 3. 多个弱符号：选择占用空间最大的弱符号。

3. 静态链接

(/)

- **定义**：链接器（ld）将多个可重定位目标文件（.o）和静态库（.a）合并，生成独立可执行文件的过程。
- **静态库（.a）**：多个相关.o 文件的归档集合（用 ar 工具创建），链接时仅复制被引用的.o 模块到可执行文件，未引用模块不复制。

静态链接核心步骤

1. **符号解析**：遍历所有输入文件的符号表，建立“符号引用 - 符号定义”的映射，确保无未定义符号（外部符号均找到定义）。
2. **重定位**：
 - **合并节**：将所有输入文件的同名节（如.text、.data）合并为一个全局节。
 - **分配地址**：为合并后的节和所有符号分配最终虚拟地址（从可执行文件的基地址开始）。
 - **修改引用**：根据.rel.text/.rel.data 中的重定位信息，将代码 / 数据中“未确定的符号地址”替换为分配后的最终地址，分为两种类型：
 - **绝对地址重定位（R_386_32）**：用于数据引用（如全局变量地址），直接替换为符号的最终虚拟地址。
 - **PC 相对地址重定位（R_386_PC32）**：用于跳转指令（call、jmp），计算“目标地址 - 重定位处下一条指令地址”，替换为偏移量。

4. 动态链接

- **定义**：程序加载时或运行时，由动态链接器（如 Linux 的 ld-linux.so）加载共享库（.so）并完成链接，而非编译时合并到可执行文件。
- **优势**：节省磁盘空间（可执行文件体积小）、节省内存（多进程共享同一.so 副本）、便于库更新（无需重新编译可执行文件）。

关键技术

- **位置无关代码（PIC - Position-Independent Code）**：共享库的代码不依赖固定地址，可加载到内存任意位置，通过以下机制实现：
 - **全局偏移量表（GOT）**：存储共享库中全局变量 / 函数的最终地址，代码通过 GOT 间接访问全局符号，GOT 在加载时由动态链接器填充地址。
 - **过程链接表（PLT）**：处理共享库函数调用，首次调用时通过 PLT 触发动态链接器解析函数地址并填入 GOT，后续调用直接通过 GOT 跳转，避免重复解析。

动态链接过程

1. **编译时**：编译器生成可执行文件时，仅记录共享库的依赖关系（如需要 libc.so），不包含共享库代码。
2. **加载时**：加载器加载可执行文件后，发现存在动态库依赖，启动动态链接器：
 - 动态链接器加载所有依赖的共享库到内存（选择未被其他进程占用的地址）。
 - 动态链接器完成符号解析（为可执行文件和共享库的未定义符号找到定义）和重定位（填充 GOT/PLT 中的地址）。
3. **运行时**：程序执行过程中，若调用共享库中未解析的函数，触发动态链接器进行“延迟绑定”（首次调用时解析地址）。

5. 可执行文件的加载

(/)

- **加载器**：操作系统内核的组件（如 Linux 的 `execve` 系统调用处理逻辑），负责将可执行文件从磁盘加载到内存并准备执行。

加载核心步骤

1. **读取文件头部**：读取 ELF 头和程序头表，确认文件格式合法、架构匹配。
2. **创建虚拟地址空间**：为进程分配独立的虚拟地址空间，划分代码段（可读可执行）、数据段（可读可写）、栈（可读可写，向低地址增长）、堆（可读可写，向高地址增长）等区域。
3. **映射文件到内存**：根据程序头表，将可执行文件的代码段、数据段“映射”到虚拟地址空间（并非立即复制所有数据到物理内存，而是利用虚拟内存的“按需调页”机制，访问时才加载对应页到物理内存）。
4. **初始化环境**：
 - 初始化栈：压入命令行参数 (`argv`)、环境变量 (`envp`)。
 - 加载动态库（若有）：启动动态链接器完成动态链接。
5. **跳转执行**：设置 CPU 的程序计数器 (EIP) 为可执行文件的入口点（`start` 函数，由编译器自动生成），`start` 函数初始化后调用 `main` 函数，程序开始执行。

进程概念

- **进程**：程序的一次执行实例，拥有独立的虚拟地址空间、寄存器状态、PCB（进程控制块）等资源，与其他进程隔离。
- **关键系统调用**：
 - `fork()`：创建子进程，子进程复制父进程的虚拟地址空间、上下文，父进程返回子进程 PID，子进程返回 0。
 - `execve()`：在当前进程中加载并执行新的可执行文件，替换当前进程的代码、数据、栈，仅保留 PID 等核心资源。

6. CPU 执行程序与异常 / 中断

指令周期

- CPU 通过“取指→译码→执行→写回→更新 PC”的循环执行程序，每个循环处理一条指令，是 CPU 的基本工作单元。

异常 (Exception)

- **定义**：CPU 内部事件，由当前指令执行引起，同步发生（与指令执行相关）。
- **分类与处理**：

异常类型	触发原因	处理后返回位置	示例
故障 (Fault)	可恢复错误（如缺页、除以零）	重新执行引起故障的指令	缺页异常（加载页后重试）
陷阱 (Trap)	有意的异常（如系统调用）	返回下一条指令	<code>int \$0x80</code> （系统调用）
中止 (Abort)	不可恢复错误（如硬件错误）	不返回，终止进程	内存校验错误

中断 (Interrupt)



- **定义**：CPU 外部事件（来自 I/O 设备），与当前指令无关，异步发生。
- **触发源**：键盘输入、磁盘 I/O 完成、定时器到期等。
- **处理过程**：与异常类似，但中断不影响当前指令执行，处理后返回下一条指令。

异常 / 中断统一处理流程

1. **检测事件**：CPU 在指令执行间隙检测异常 / 中断信号。
2. **保存现场**：将当前 EIP（下一条指令地址）、EFLAGS（标志寄存器）压入内核栈，保护当前执行上下文。
3. **查找处理程序**：根据异常 / 中断号（0-255），在中断描述符表（IDT）中查找对应的中断服务程序（ISR）地址。
4. **执行处理程序**：切换到内核模式，执行 ISR（如缺页处理、I/O 完成处理）。
5. **恢复现场**：从内核栈弹出 EIP、EFLAGS，恢复用户模式，跳回被打断的指令（故障）或下一条指令（陷阱 / 中断）继续执行。

7. 指令流水线（Pipeline）

- **目的**：通过“重叠执行指令的不同阶段”提高 CPU 吞吐率（理想情况下每个时钟周期完成一条指令）。
- **经典五级流水线**：取指（IF）→ 译码（ID）→ 执行（EX）→ 访存（MEM）→ 写回（WB）。
 - 示例：第 1 条指令执行 EX 阶段时，第 2 条指令执行 ID 阶段，第 3 条指令执行 IF 阶段，实现并行处理。

流水线冒险（Hazard）

- **定义**：阻碍指令按预期周期执行的问题，需通过硬件或软件优化解决。
- **分类**：
 1. **结构冒险**：硬件资源冲突（如同一个时钟周期内两条指令都需使用 ALU），解决方法：增加资源副本、资源分时复用。
 2. **数据冒险**：指令依赖前一条指令的结果（如 ADD EAX, EBX 后立即使用 EAX），解决方法：数据前推（Forwarding，直接传递中间结果）、插入气泡（NOP，等待结果生成）。
 3. **控制冒险**：分支指令（如 jcc）导致流水线不知道下一条指令地址，解决方法：分支预测（如静态预测、动态预测）、延迟分支。

markdown
通常由多个功能相关的“节（Section）”组成（如.text 和.rodata 合并为代码段）。

2. 符号和符号解析

- **符号**：程序中的变量名、函数名等标识，是模块间链接的“桥梁”。
- **符号表（.symtab）**：记录每个符号的属性，包括符号名（索引指向.strtab）、符号类型（全局 / 外部 / 本地）、符号所在节（如.text/.data）、符号地址 / 偏移量、符号大小。

符号分类

符号类型	定义位置	可见范围	示例
全局符号	本模块	本模块及其他模块	非 static 的全局变量、函数
外部符号	其他模块	仅本模块引用	extern 声明的变量 / 函数

符号类型	定义位置	可见范围	示例
(本地符号	本模块	仅本模块内部	static 的变量 / 函数

符号解析规则 (Linux 环境)

- **强符号与弱符号：**
 - 强符号：已初始化的全局变量、函数名（不可重复定义）。
 - 弱符号：未初始化的全局变量（可重复定义）。
- **多重定义处理：**
 1. 不允许多个同名强符号（链接错误）。
 2. 一个强符号 + 多个弱符号：以强符号为准。
 3. 多个弱符号：选择占用空间最大的弱符号。

3. 静态链接

- **定义：**链接器 (ld) 将多个可重定位目标文件 (.o) 和静态库 (.a) 合并，生成独立可执行文件的过程。
- **静态库 (.a)：**多个相关.o 文件的归档集合（用 ar 工具创建），链接时仅复制被引用的.o 模块到可执行文件，未引用模块不复制。

静态链接核心步骤

1. **符号解析：**遍历所有输入文件的符号表，建立“符号引用 - 符号定义”的映射，确保无未定义符号（外部符号均找到定义）。
2. **重定位：**
 - 合并节：将所有输入文件的同名节（如.text、.data）合并为一个全局节。
 - 分配地址：为合并后的节和所有符号分配最终虚拟地址（从可执行文件的基地址开始）。
 - 修改引用：根据.rel.text/.rel.data 中的重定位信息，将代码 / 数据中“未确定的符号地址”替换为分配后的最终地址，分为两种类型：
 - 绝对地址重定位 (R_386_32)：用于数据引用（如全局变量地址），直接替换为符号的最终虚拟地址。
 - PC 相对地址重定位 (R_386_PC32)：用于跳转指令 (call、jmp)，计算“目标地址 - 重定位处下一条指令地址”，替换为偏移量。

4. 动态链接

- **定义：**程序加载时或运行时，由动态链接器（如 Linux 的 ld-linux.so）加载共享库 (.so) 并完成链接，而非编译时合并到可执行文件。
- **优势：**节省磁盘空间（可执行文件体积小）、节省内存（多进程共享同一.so 副本）、便于库更新（无需重新编译可执行文件）。

关键技术

- **位置无关代码 (PIC - Position-Independent Code)：**共享库的代码不依赖固定地址，可加载到内存任意位置，通过以下机制实现：



- 全局偏移量表（GOT）：存储共享库中全局变量 / 函数的最终地址，代码通过 GOT 间接访问全局符号，GOT 在加载时由动态链接器填充地址。
- 过程链接表（PLT）：处理共享库函数调用，首次调用时通过 PLT 触发动态链接器解析函数地址并填入 GOT，后续调用直接通过 GOT 跳转，避免重复解析。

动态链接过程

1. **编译时**：编译器生成可执行文件时，仅记录共享库的依赖关系（如需要 libc.so），不包含共享库代码。
2. **加载时**：加载器加载可执行文件后，发现存在动态库依赖，启动动态链接器：
 - 动态链接器加载所有依赖的共享库到内存（选择未被其他进程占用的地址）。
 - 动态链接器完成符号解析（为可执行文件和共享库的未定义符号找到定义）和重定位（填充 GOT/PLT 中的地址）。
3. **运行时**：程序执行过程中，若调用共享库中未解析的函数，触发动态链接器进行“延迟绑定”（首次调用时解析地址）。

5. 可执行文件的加载

- **加载器**：操作系统内核的组件（如 Linux 的 `execve` 系统调用处理逻辑），负责将可执行文件从磁盘加载到内存并准备执行。

加载核心步骤

1. **读取文件头部**：读取 ELF 头和程序头表，确认文件格式合法、架构匹配。
2. **创建虚拟地址空间**：为进程分配独立的虚拟地址空间，划分代码段（可读可执行）、数据段（可读可写）、栈（可读可写，向低地址增长）、堆（可读可写，向高地址增长）等区域。
3. **映射文件到内存**：根据程序头表，将可执行文件的代码段、数据段“映射”到虚拟地址空间（并非立即复制所有数据到物理内存，而是利用虚拟内存的“按需调页”机制，访问时才加载对应页到物理内存）。
4. **初始化环境**：
 - 初始化栈：压入命令行参数（`argv`）、环境变量（`envp`）。
 - 加载动态库（若有）：启动动态链接器完成动态链接。
5. **跳转执行**：设置 CPU 的程序计数器（EIP）为可执行文件的入口点（`start` 函数，由编译器自动生成），`start` 函数初始化后调用 `main` 函数，程序开始执行。

进程概念

- **进程**：程序的一次执行实例，拥有独立的虚拟地址空间、寄存器状态、PCB（进程控制块）等资源，与其他进程隔离。
- **关键系统调用**：
 - `fork()`：创建子进程，子进程复制父进程的虚拟地址空间、上下文，父进程返回子进程 PID，子进程返回 0。
 - `execve()`：在当前进程中加载并执行新的可执行文件，替换当前进程的代码、数据、栈，仅保留 PID 等核心资源。

6. CPU 执行程序与异常 / 中断

(/) 指令周期

- CPU 通过“取指→译码→执行→写回→更新 PC”的循环执行程序，每个循环处理一条指令，是 CPU 的基本工作单元。

异常 (Exception)

- 定义：CPU 内部事件，由当前指令执行引起，同步发生（与指令执行相关）。
- 分类与处理：

异常类型	触发原因	处理后返回位置	示例
故障 (Fault)	可恢复错误（如缺页、除以零）	重新执行引起故障的指令	缺页异常（加载页后重试）
陷阱 (Trap)	有意的异常（如系统调用）	返回下一条指令	int \$0x80（系统调用）
中止 (Abort)	不可恢复错误（如硬件错误）	不返回，终止进程	内存校验错误

中断 (Interrupt)

- 定义：CPU 外部事件（来自 I/O 设备），与当前指令无关，异步发生。
- 触发源：键盘输入、磁盘 I/O 完成、定时器到期等。
- 处理过程：与异常类似，但中断不影响当前指令执行，处理后返回下一条指令。

异常 / 中断统一处理流程

1. 检测事件：CPU 在指令执行间隙检测异常 / 中断信号。
2. 保存现场：将当前 EIP（下一条指令地址）、EFLAGS（标志寄存器）压入内核栈，保护当前执行上下文。
3. 查找处理程序：根据异常 / 中断号（0₂₅₅），在中断描述符表（IDT）中查找对应的中断服务程序（ISR）地址。
4. 执行处理程序：切换到内核模式，执行 ISR（如缺页处理、I/O 完成处理）。
5. 恢复现场：从内核栈弹出 EIP、EFLAGS，恢复用户模式，跳回被打断的指令（故障）或下一条指令（陷阱 / 中断）继续执行。

7. 指令流水线 (Pipeline)

- 目的：通过“重叠执行指令的不同阶段”提高 CPU 吞吐率（理想情况下每个时钟周期完成一条指令）。
- 经典五级流水线：取指 (IF) → 译码 (ID) → 执行 (EX) → 访存 (MEM) → 写回 (WB)。
 - 示例：第 1 条指令执行 EX 阶段时，第 2 条指令执行 ID 阶段，第 3 条指令执行 IF 阶段，实现并行处理。

流水线冒险 (Hazard)

- 定义：阻碍指令按预期周期执行的问题，需通过硬件或软件优化解决。
- 分类：
 1. 结构冒险：硬件资源冲突（如同一时钟周期内两条指令都需使用 ALU），解决方法：增加资源副本、资源分时复用。



- 2. **数据冒险**：指令依赖前一条指令的结果（如 ADD EAX, EBX 后立即使用 EAX），解决方法：数据前推（Forwarding，直接传递中间结果）、插入气泡（NOP，等待结果生成）。
- 3. **控制冒险**：分支指令（如 jcc）导致流水线不知道下一条指令地址，解决方法：分支预测（如静态预测、动态预测）、延迟分支。

第五章：程序的存储访问

1. 存储器层次结构（Memory Hierarchy）

- **核心目的**：平衡存储器的速度、容量、成本矛盾，构建“速度接近最快层、容量接近最慢层、成本接近最慢层”的存储系统。

层次结构（从上层到下层）

存储层次	存储介质	速度	容量	成本（每GB）	访问方式
CPU 寄存器	寄存器电路	最快（ns 级）	最小（KB 级）	最高	随机访问
高速缓存（Cache）	SRAM	快（ns 级）	小（MB 级）	高	随机访问
主存（RAM）	DRAM	中（ns 级）	中（GB 级）	中	随机访问
本地二级存储	磁盘 / SSD	慢（ms 级）	大（TB 级）	低	磁盘：直接访问；SSD：随机访问
远程二级存储	网络存储	最慢（s 级）	最大	最低	网络传输

核心原理：程序访问的局部性

- **时间局部性**：最近访问过的内存单元，短期内很可能再次被访问（如循环变量、频繁调用的函数）。
- **空间局部性**：访问某个内存单元后，其相邻单元很可能被访问（如数组遍历、顺序执行的指令）。
- **工作方式**：上层存储器是下层存储器的“缓存”，数据在相邻层之间以“块（Block）”为单位传输（如 Cache 与主存以 64B 块传输，主存与磁盘以 4KB 页传输）。

2. 主存（Main Memory）

- **核心介质**：DRAM（动态随机存取存储器），需周期性刷新（每 64ms）以保持数据，速度比 SRAM 慢，但密度高、成本低。

主存关键技术

- **SDRAM（同步 DRAM）**：与系统时钟同步，支持“突发传输（Burst Transfer）”，一次地址请求后连续传输多个数据（如连续读取 4 个 64B 块），利用空间局部性提升带宽。
- **DDR SDRAM（双倍速率 SDRAM）**：在时钟上升沿和下降沿都传输数据，带宽比 SDRAM 翻倍，后续 DDR2/DDR3/DDR4 通过提升频率、增加预取位数进一步提升带宽（如 DDR4 预取 8 位，DDR5 预取 16 位）。
- **内存条与通道**：



- 内存条（DIMM）：多个 DRAM 芯片集成在 PCB 板上，如 DDR4 DIMM、DDR5 DIMM。
- 多通道技术：CPU 通过多个内存通道并行访问多个内存条（如双通道、四通道），总带宽 = 单通道带宽 × 通道数（如双通道 DDR4-3200 带宽 = 2×25.6GB/s=51.2GB/s）。

主存访问流程

1. CPU 通过内存控制器发送地址和控制信号（读 / 写）。
2. 内存控制器将地址解析为 DRAM 芯片的“行地址（RAS）”和“列地址（CAS）”，先激活行，再读取列数据。
3. 数据通过数据总线传输到 CPU（读操作）或从 CPU 写入 DRAM（写操作）。

3. 高速缓存（Cache）- 核心重点

- 目的：弥补 CPU 与主存的速度差距（CPU 速度是主存的 100₁₀₀₀ 倍），通过缓存主存中“频繁访问的数据块”，减少 CPU 访存等待时间。

基本概念

- 块（Block）：Cache 与主存之间的数据传输单位（如 64B），主存和 Cache 均划分为大小相等的块。
- Cache 行（Line）：Cache 中存储一个主存块的空间，包含三部分：
 - 数据块：主存块的副本。
 - 标记（Tag）：主存块地址的高位，用于标识当前行存储的是哪个主存块。
 - 有效位（Valid Bit）：指示当前行的数据是否有效（初始化时为 0，加载主存块后为 1）。
- 命中与缺失：
 - 命中（Hit）：CPU 访问的地址对应的主存块已在 Cache 中，直接从 Cache 读取数据，耗时为“命中时间（Hit Time，通常 1₃ 个时钟周期）”。
 - 缺失（Miss）：CPU 访问的主存块不在 Cache 中，需从主存加载块到 Cache，额外耗时为“缺失代价（Miss Penalty，通常 10₁₀₀ 个时钟周期）”。
- 核心公式：平均访存时间 = 命中时间 + 缺失率 × 缺失代价（缺失率 = 1 - 命中率）。

映射方式（主存块到 Cache 行的对应规则）

映射方式	规则	地址划分	优点	缺点	应用场景
直接映射	主存块号 mod Cache 总行数 = Cache 行号（每个主存块对应唯一 Cache 行）	Tag + 行索引（Index） + 块内偏移（Offset）	实现简单、命中时间短	冲突缺失率高（多个主存块竞争同一行）	早期 CPU L1 Cache
全相联映射	主存块可映射到任意 Cache 行	Tag + 块内偏移	冲突缺失率最低	实现复杂（需并行比较所有 Tag）、命中时间长	小容量 Cache（如 TLB）
组相联映射	Cache 行分“组（Set）”，主存块号 mod Cache 总组数 = 组号，块可映射到组内任意行	Tag + 组索引 + 块内偏移	平衡性能与复杂度	实现成本高于直接映射	现代 CPU L1/L2/L3 Cache

- < (/)
- **N 路组相联**：每组包含 N 个 Cache 行（如 4 路组相联 = 每组 4 行），N 越大，冲突缺失率越低，但实现复杂度越高。

替换算法（Cache 缺失且组 / 行已满时，选择替换的旧块）

- **LRU (Least Recently Used, 最近最少使用)**：替换组内“最长时间未被访问”的块，符合局部性原理，性能最好，但需硬件记录访问时间（如使用计数器、栈结构）。
- **FIFO (First-In-First-Out, 先进先出)**：替换组内“最早进入”的块，实现简单（用队列记录顺序），但可能替换仍需使用的块，性能略差于 LRU。
- **Random (随机)**：随机选择组内一块替换，实现最简单，性能接近 LRU（尤其在 N 较大时），部分 CPU 采用。

写策略（CPU 写 Cache 时，如何保持 Cache 与主存一致性）

写策略	规则	优点	缺点	搭配的“写缺失处理”
写直达 (Write-Through)	写 Cache 的同时，立即写主存	一致性好 (Cache 与主存始终同步)、实现简单	写操作慢（每次写需访问主存）、总线带宽占用高	非写分配（写缺失时直接写主存，不加载块到 Cache）
写回 (Write-Back)	仅写 Cache，不立即写主存；为 Cache 行增加“修改位 (Dirty Bit)”，仅当脏行被替换时，才写回主存	写操作快（仅访问 Cache）、减少总线访问	一致性维护复杂（存在 Cache 与主存不同步时期）	写分配（写缺失时加载块到 Cache，再写 Cache）

4. 虚拟存储器（Virtual Memory） - 核心重点

- **目的**：
 1. 提供“远超物理内存大小”的虚拟地址空间，支持运行比物理内存大的程序。
 2. 简化内存管理：每个进程拥有独立虚拟地址空间，地址连续，无需关心物理内存碎片。
 3. 内存保护：通过页表项权限位（读 / 写 / 执行）限制进程对内存的访问范围，防止越权。

核心概念

- **虚拟地址 (VA)**：CPU 发出的地址，属于进程私有虚拟地址空间（如 32 位系统为 4GB，64 位系统为 2^{64} 字节）。
- **物理地址 (PA)**：主存的实际地址，所有进程共享物理地址空间（大小由物理内存容量决定）。
- **页 (Page)**：虚拟地址空间划分的固定大小块（如 4KB、2MB），是虚拟内存的基本单位。
- **页框 (Page Frame)**：物理地址空间划分的块，大小与页相同，用于存放一个虚拟页的内容。
- **页表 (Page Table)**：操作系统为每个进程维护的“虚拟页→物理页框”映射表，存储在主存中，由页表项 (PTE) 组成。
- **MMU (内存管理单元)**：CPU 中的硬件模块，负责将虚拟地址实时翻译为物理地址。

页表项 (PTE) 结构

一个典型的 PTE 包含以下关键字段：



- **有效位 (Valid Bit)** : 1 = 虚拟页在主存中 (映射有效) ; 0 = 虚拟页不在主存中 (在磁盘上, 映射无效)。
- **物理页框号 (PPN)** : 虚拟页对应的物理页框地址 (有效位为 1 时有效)。
- **磁盘地址**: 虚拟页在磁盘上的位置 (有效位为 0 时, 用于缺页时加载)。
- **权限位 (Read/Write/Execute)** : 限制对该页的访问权限 (如只读、可写、可执行)。
- **修改位 (Dirty Bit)** : 1 = 该页在主存中被修改过; 0 = 未修改, 用于页替换时判断是否需要写回磁盘。
- **访问位 (Accessed Bit)** : 1 = 该页最近被访问过; 0 = 未访问, 用于页面替换算法 (如 LRU)。

地址翻译过程

1. **拆分虚拟地址**: MMU 将虚拟地址 (VA) 拆分为“虚拟页号 (VPN)”和“页内偏移 (VPO)”, 页内偏移大小由页大小决定 (如 4KB 页的偏移为 12 位, $2^{12}=4096$)。
2. **查找页表**: MMU 以 VPN 为索引, 访问页表 (页表基地址由“页表基址寄存器 (PTBR)”指示), 找到对应的 PTE。
3. **判断映射有效性**:
 - **命中 (有效位 = 1)** : 从 PTE 中取出物理页框号 (PPN), 与页内偏移 (VPO) 拼接, 得到物理地址 (PA), CPU 用 PA 访问主存。
 - **缺页 (有效位 = 0)** : 触发“缺页异常 (Page Fault)”, 由操作系统处理。

缺页处理流程

1. **中断进程**: 操作系统暂停当前进程的执行, 保存进程上下文 (寄存器、PC 等)。
2. **定位磁盘地址**: 根据 PTE 中的磁盘地址, 找到虚拟页在磁盘上的存储位置 (如交换分区、可执行文件)。
3. **分配物理页框**:
 - 若主存仍有空闲页框, 直接分配;
 - 若主存已满, 执行“页面替换算法” (如 LRU、Clock 算法) 选择一个“牺牲页框”, 若牺牲页框的修改位为 1, 先将其内容写回磁盘。
4. **加载页面**: 将磁盘上的虚拟页数据读入分配的物理页框。
5. **更新页表**: 修改对应的 PTE (有效位设为 1, 填入物理页框号, 重置修改位和访问位)。
6. **恢复进程**: 恢复被中断进程的上下文, 重新执行导致缺页的指令 (此时映射已有效, 可正常访问)。

TLB (Translation Lookaside Buffer - 快表)

- **问题**: 页表存储在主存中, 每次地址翻译需访问主存 (查找 PTE), 导致 CPU 访存次数加倍 (先查页表, 再访数据), 速度慢。
- **解决**: TLB 是“页表项的高速缓存”, 存储最近使用的 VPN→PTE 映射, 集成在 CPU 中, 访问速度接近寄存器。
- **TLB 工作流程**:
 1. CPU 发出 VA, MMU 先以 VPN 为索引查找 TLB。
 2. **TLB 命中**: 直接从 TLB 中获取 PTE, 完成地址翻译 (无需访问主存页表)。
 3. **TLB 缺失**: 访问主存中的页表找到 PTE, 完成地址翻译, 同时将该 PTE 存入 TLB (可能替换旧的 TLB 项, 遵循 LRU 等算法)。

第六章：程序中 I/O 操作的实现

(/)

1. I/O 子系统组成

- **I/O 设备**：计算机与外部交互的硬件，按传输单位分为两类：
 - 字符设备：以字符为单位传输，不可寻址（如键盘、鼠标、串口）。
 - 块设备：以固定大小块（如 512B、4KB 扇区）传输，可寻址（如磁盘、SSD、U 盘）。
- **设备控制器**：I/O 设备与系统总线的接口，包含：
 - 数据寄存器：暂存 CPU 与设备间传输的数据。
 - 状态寄存器：指示设备当前状态（如“就绪”“忙”“错误”）。
 - 控制寄存器：CPU 写入控制命令（如“读”“写”“启动设备”）。
- **I/O 软件**：从用户层到内核层的软件集合，负责屏蔽硬件细节，提供统一接口。

2. I/O 软件层次（从上层到下层）

层次	功能	示例
用户层 I/O 软件	提供易用的应用接口，处理缓冲、格式转换	C 标准库（stdio.h）的 printf、fread、fopen
设备无关 I/O 软件	提供统一 I/O 接口，处理设备分配、缓冲管理、错误报告	操作系统的文件系统（如 ext4）、VFS
设备驱动程序	与特定设备控制器交互，将上层请求转换为硬件命令	磁盘驱动、键盘驱动、显卡驱动
中断处理程序	处理设备触发的中断，完成 I/O 完成通知、数据传输等	磁盘 I/O 完成中断处理程序、键盘中断处理程序

3. 用户空间 I/O 操作

文件抽象

- **UNIX/Linux 设计哲学**：“一切皆文件”，将普通文件、目录、I/O 设备均抽象为“文件”，用统一的文件操作接口（open、read、write、close）访问，简化编程。
- **文件描述符（File Descriptor, fd）**：内核为每个进程维护“文件描述符表”，open 文件时返回一个非负整数（fd），作为后续操作的标识；默认 fd：0（标准输入 stdin）、1（标准输出 stdout）、2（标准错误 stderr）。

核心 I/O 接口

- **系统调用接口**：用户程序通过系统调用请求内核执行 I/O 操作，直接与内核交互，无缓冲（或仅内核缓冲）：
 - open(const char *path, int flags, mode_t mode)：打开文件，返回 fd。
 - read(int fd, void *buf, size_t count)：从 fd 读取 count 字节到 buf，返回实际读取字节数。

<
(/)

- `write(int fd, const void *buf, size_t count)` : 将 `buf` 中 `count` 字节写入 `fd`, 返回实际写入字节数。
- `close(int fd)` : 关闭文件, 释放 `fd`。
- **C 标准 I/O 库 (`stdio.h`)** : 基于系统调用实现, 提供更高层接口, 自带用户级缓冲, 减少系统调用次数 (提升效率) :
 - 缓冲类型:
 - 全缓冲: 缓冲区满 (如 4KB) 或 `fflush` 时才执行实际 I/O (普通文件默认)。
 - 行缓冲: 遇到换行符 (`\n`) 或缓冲区满时执行 I/O (终端默认, 如 `printf`)。
 - 无缓冲: 立即执行 I/O (如 `stderr`, 确保错误信息及时输出)。
 - 核心函数: `fopen` (打开文件, 返回 `FILE*`)、`fread`、`fwrite`、`printf`、`scanf`、`fclose`。

4. 内核空间 I/O 操作

VFS (虚拟文件系统)

- **目的**: 屏蔽不同文件系统 (如 `ext4`、`NTFS`、`FAT32`) 和设备的差异, 为上层提供统一的文件操作接口 (如 `read`、`write`)。
- **核心数据结构**:
 - `inode` (索引节点): 存储文件元数据 (大小、权限、修改时间、数据块指针), 每个文件对应一个 `inode`。
 - `dentry` (目录项): 存储文件名与 `inode` 的映射, 构成文件系统的目录树。

内核缓冲 (Page Cache/Buffer Cache)

- **目的**: 利用主存作为磁盘的缓存, 减少实际磁盘 I/O 次数 (磁盘速度远慢于主存)。
- **Page Cache**: 缓存文件数据 (以页为单位), 用户程序读 / 写文件时, 先访问 `Page Cache`, 命中则直接操作, 缺失则从磁盘加载到 `Page Cache`。
- **Buffer Cache**: 缓存磁盘块数据 (以扇区为单位), 用于块设备的直接 I/O (如磁盘分区访问), 现代 Linux 已将 `Buffer Cache` 整合到 `Page Cache` 中。

设备驱动程序

- **角色**: 内核与硬件设备的“翻译官”, 将上层抽象的 I/O 请求 (如“读磁盘第 100 扇区”) 转换为设备控制器能理解的硬件命令 (如写入设备控制寄存器的命令字)。
- **核心功能**:
 1. 初始化设备: 驱动加载时初始化设备控制器 (如设置中断、配置寄存器)。
 2. 处理 I/O 请求: 响应上层 (如文件系统) 的 I/O 请求, 向设备发送命令, 等待设备完成。
 3. 处理中断: 设备完成 I/O 后触发中断, 驱动的中断处理程序读取数据 / 状态, 通知上层请求完成。

5. I/O 控制方式 (重要)

1. 程序直接控制 (轮询 / Polling)

- **原理**: CPU 主动循环检查设备状态寄存器, 直到设备就绪, 再执行数据传输。



- **流程：**
 1. CPU 向设备控制器发送“读 / 写”命令。
 2. CPU 循环读取设备状态寄存器，判断设备是否“就绪”（忙等）。
 3. 设备就绪后，CPU 直接读写数据寄存器，完成数据传输。
- **优点：**实现简单，无需中断硬件支持。
- **缺点：**CPU “忙等”期间无法执行其他任务，CPU 利用率极低，仅适用于慢速、低频率设备（如早期打印机）。

2. 中断驱动 I/O (Interrupt-driven I/O)

- **原理：**CPU 启动 I/O 操作后，转去执行其他任务；设备完成操作后触发中断，CPU 响应中断并处理数据传输。
- **流程：**
 1. CPU 向设备控制器发送“读 / 写”命令，设置设备中断使能。
 2. CPU 返回，继续执行其他任务（如用户程序）。
 3. 设备完成数据传输后，向 CPU 发送中断请求。
 4. CPU 暂停当前任务，保存上下文，执行设备驱动的中断处理程序（读取数据 / 状态，通知上层）。
 5. 中断处理完成后，CPU 恢复上下文，继续执行原任务。
- **优点：**CPU 无需忙等，利用率大幅提升。
- **缺点：**每次数据传输（如 1 字节）都需触发一次中断，中断处理有开销（保存 / 恢复上下文），不适用于高速、大量数据传输（如磁盘、网络）。

3. DMA 控制 (Direct Memory Access)

- **原理：**引入 DMA 控制器 (DMAC)，代替 CPU 控制“设备与主存”之间的直接数据传输，CPU 仅在传输开始和结束时介入。
- **流程：**
 1. CPU 向 DMAC 发送“传输请求”，包含：设备地址（数据寄存器地址）、主存地址（数据缓冲区地址）、传输长度、传输方向（设备→主存 / 主存→设备）。
 2. CPU 返回，执行其他任务；DMAC 向设备控制器发送命令，启动数据传输。
 3. DMAC 直接控制设备与主存之间的数据传输（无需 CPU 参与），传输 1 字节后自动递增地址、递减长度，直到传输完成。
 4. DMAC 向 CPU 发送“传输完成”中断，CPU 执行中断处理程序（如通知上层数据已就绪）。
- **优点：**CPU 几乎完全解放，仅在传输开始和结束时处理，适合高速、大量数据传输（如磁盘、SSD、网络接口），是现代计算机的主流 I/O 控制方式。
- **缺点：**需要额外的 DMAC 硬件，实现复杂度高于前两种方式。

6. I/O 端口编址

- **目的：**CPU 通过“端口地址”访问设备控制器的寄存器（数据、状态、控制寄存器），需为端口分配唯一地址。
- **两种编址方式：**

< (/)	编址方式	规则	访问指令	优点	缺点	应用架构
	独立编址 (I/O 映射)	I/O 端口拥有独立地址空间，与内存地址空间不重叠	专用 I/O 指令（如 x86 的 in、out）	内存地址空间不受影响	需专用指令，地址空间有限	x86 架构
	统一编址 (内存映射)	I/O 端口与内存单元共享同一地址空间，端口地址是内存地址的一部分	普通访存指令（mov、load、store）	无需专用指令，地址空间大	占用部分内存地址空间	ARM、MIPS、RISC-V 等 RISC 架构
回复						

 Chaos (/u/1) 于 2025年10月25日 更改标题为「计算机系统原理 - 考前冲刺核心知识点」

说点什么吧...